C++ COMPLETE REFERENCE LIBRARY • MODULE 19

19 NAMESPACES AND PREPROCESSOR — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 namespaces and scope resolution
3. **Theory Chapter:** 02 preprocessor directives and macro pitfalls
4. **Theory Chapter:** 03 header guards and translation units
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

19 — Namespaces and the Preprocessor

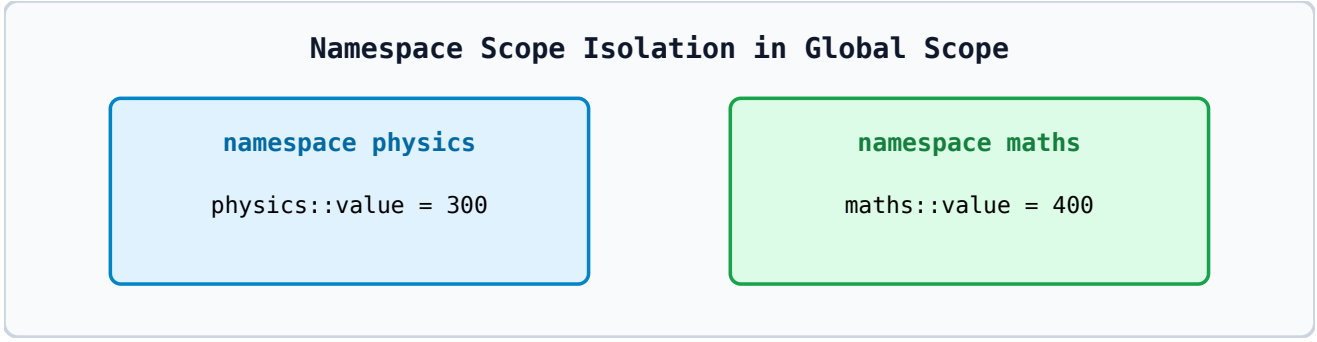
This module covers declaring custom `namespace` scopes, resolving name collisions with `::`, `using` declarations vs. `using` directives, anonymous namespaces, `#include` header guards, and why `constexpr` / `inline` functions should replace `#define` macros.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_namespaces_and_scope_resolution.md` (./01_THEORY/01_namespaces_and_scope_resolution.md)
  - `02_preprocessor_directives_and_macro_pitfalls.md` (./01_THEORY/02_preprocessor_directives_and_macro_pitfalls.md)
  - `03_header_guards_and_translation_units.md` (./01_THEORY/03_header_guards_and_translation_units.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_custom_namespaces_resolution.cpp` (./02_PROGRAMS/01_custom_namespaces_resolution.cpp) (Category A)

  - `02_macro_pitfall_vs_constexpr_inline.cpp` (`./02_PROGRAMS/02_macro_pitfall_vs_constexpr_inline.cpp`) (Category A)
  - `03_namespace_collision_compile_error.cpp` (`./02_PROGRAMS/03_namespace_collision_compile_error.cpp`) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_namespace_scopes.svg` (`./05_DIAGRAMS/diagram_01_namespace_scopes.svg`)

01 — Namespaces, `using` Declarations, and `using` Directives



Namespace Scopes

- **Namespace Definition**: Groups identifiers (`classes` , `functions` , `constants`) into a named scope (`namespace math_dept { int code = 101; }`) to prevent global naming collisions.
- **Explicit Qualification**: `math_dept::code`
- `using` **Declaration**: `using math_dept::code;` imports only that single name.
- `using` **Directive**: `using namespace math_dept;` makes all names in `math_dept` visible.

02 — Preprocessor Macros (`#define`) vs. `constexpr` and `inline`

Why `#define SQUARE(x) x * x` is Dangerous

1. **Operator Precedence Bug**: `SQUARE(3 + 2)` expands textually to `3 + 2 * 3 + 2 = 11` instead of `25`!
2. **Multiple Evaluation Bug**: Even with parentheses `#define SQUARE(x) ((x) * (x))`, calling `SQUARE(++a)` expands to `((++a) * (++a))` (Undefined Behavior!).
3. **Modern Fix**: Use `constexpr int square(int x) { return x * x; }`.

03 — Header Include Guards (`#ifndef` / `#define` / `#endif`)

Prevent a header file from being included multiple times in the same `.cpp` translation unit (which would violate the One Definition Rule):

CPP

```cpp
#ifndef PERSON_H
#define PERSON_H
// declarations
#endif
```

Visual Concept & Memory Diagrams (19 NAMESPACES AND PREPROCESSOR)

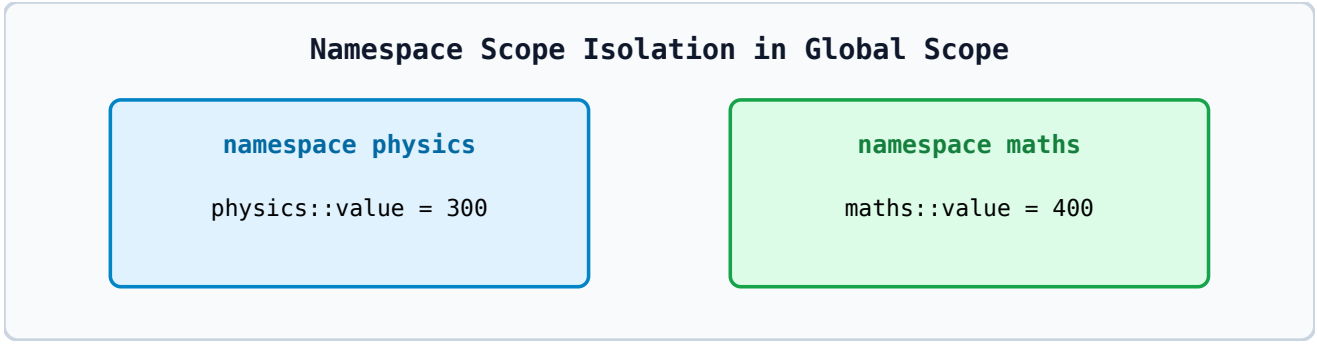


Diagram: diagram 01 namespace scopes

Annotated C++ Programs (19 NAMESPACES AND PREPROCESSOR)

Program: `01_custom_namespaces_resolution.cpp`

C++ SOURCE FILE • 01_custom_namespaces_resolution.cpp

```cpp
// Program Title: Resolving Identical Identifiers Using Custom Namespaces
// Topic: Namespaces and Preprocessor
// Purpose: Show two namespaces defining 'value' and accessing both via scope resolution
'::'.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_namespaces_and_scope_resolution.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

namespace physics {
    int value = 300;
}

namespace maths {
    int value = 400;
}

int main() {
    cout << "Physics value: " << physics::value << endl;
    cout << "Maths value: " << maths::value << endl;
    return 0;
}

/*
Expected Deterministic Output:
Physics value: 300
Maths value: 400
*/
```

Program: `02_macro_pitfall_vs_constexpr_inline.cpp`

C++ SOURCE FILE • 02_macro_pitfall_vs_constexpr_inline.cpp

```cpp
// Program Title: Textual Macro Expansion Pitfall vs. constexpr Function
// Topic: Namespaces and Preprocessor — Macros vs. constexpr
// Purpose: Contrast #define SQUARE(x) x*x on (3+2) against constexpr square(3+2).
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_preprocessor_directives_and_macro_pitfalls.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

#define UNSAFE_SQUARE(x) x * x

constexpr int safe_square(int x) {
    return x * x;
}

int main() {
    cout << "UNSAFE_SQUARE(3 + 2) expands to 3 + 2 * 3 + 2 = " << UNSAFE_SQUARE(3 + 2) <<
endl;
    cout << "safe_square(3 + 2) evaluates (5 * 5) = " << safe_square(3 + 2) << endl;
    return 0;
}

/*
Expected Deterministic Output:
UNSAFE_SQUARE(3 + 2) expands to 3 + 2 * 3 + 2 = 11
safe_square(3 + 2) evaluates (5 * 5) = 25
*/
```

Program: `03_namespace_collision_compile_error.cpp`

C++ SOURCE FILE • 03_namespace_collision_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Ambiguous Lookup When Importing Two Namespaces With Identical Names
// Topic: Namespaces and Preprocessor — using Directive Collision
// Purpose: Show why bringing two namespaces with 'show()' into scope causes an ambiguity
error on unqualified 'show()'.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/01_namespaces_and_scope_resolution.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

namespace first {
    void show() { cout << "first
"; }
}

namespace second {
    void show() { cout << "second
"; }
}

using namespace first;
using namespace second;

int main() {
    show(); // COMPILE ERROR: reference to 'show' is ambiguous
    return 0;
}

/*
Why Compilation Fails:
Unqualified lookup for 'show()' finds both 'first::show()' and 'second::show()'.
Fix: Qualify explicitly as 'first::show();' and 'second::show();'.
*/
```

Conceptual Questions — Module 19: Namespaces and Preprocessor

Q1. Differentiate between a `using` declaration (`using std::cout;`) and a `using` directive (`using namespace std;`). **Q2.** Why are `constexpr` functions safer than `#define` function-like macros?

Debugging Questions — Module 19

Q4. How do you access a global variable `int x = 100;` inside `main()` when a local variable `int x = 5;` shadows it?

Predict the Output Questions — Module 19

Q3. Predict the output of `#define CUBE(x) x*x*x` when called as `CUBE(1 + 2)`.

Programming Questions — Module 19

Q5. Write a C++23 program demonstrating nested namespaces (`namespace university::computing`).

Detailed Solutions — Conceptual Questions (Module 19)

- **Q1 Solution**: A `using` declaration imports only one specific identifier into the current scope, whereas a `using` directive makes every identifier in the entire namespace visible.
- **Q2 Solution**: `constexpr` functions respect type checking, scope, and evaluate their arguments once, avoiding macro precedence and multiple-evaluation bugs.

Detailed Solutions — Predict the Output & Debugging (Module 19)

- **Q3 Solution**: `CUBE(1 + 2)` expands to `1 + 2 * 1 + 2 * 1 + 2 = 1 + 2 + 2 + 2 = 7` (instead of `27`).
- **Q4 Solution**: Use the unary global scope resolution operator: `::x`.

Detailed Solutions — Programming Questions (Module 19)

CPP

```cpp
#include <iostream>
using namespace std;
namespace university::computing {
    int credits = 4;
}
int main() {
    cout << "Credits: " << university::computing::credits << endl;
    return 0;
}
```